MODULE

04

Branches & merging

Build features in parallel without breaking main, then bring the work together.

YOU WILL LEARN

Branches are pointers	`01_branches_are_pointers.md`
Fast-forward vs three-way	`02_fast_forward_and_three_way.md`
Merge conflicts	`03_merge_conflicts.md`
Exercise: resolve a conflict	`ex04_merge_conflict/`
Interview Q&A	`INTERVIEW.md`

WHY BRANCHES

Why branches?

THE PROBLEM

You’re halfway through a big feature when a production bug comes in. Your half-done code can’t be released, and five teammates are changing the same project at the same time.

THE IDEA

A branch is a separate line of work. main stays releasable; each feature or fix lives on its own branch until it’s merged.

PICTURE IT BRANCHES

A branch is a sticky note on a commit

📌 Commits form a timeline. A branch is just a **sticky note** with a commit ID on it; creating one copies nothing. **HEAD** is the note you're holding: new commits move that note forward.

`main`
points to the release commit

→

`feature/search`
points to your newest commit

→

`HEAD`
points to the branch you're on

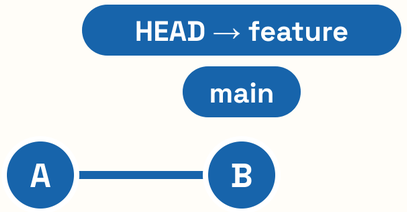
→

`new commit`
moves your branch forward

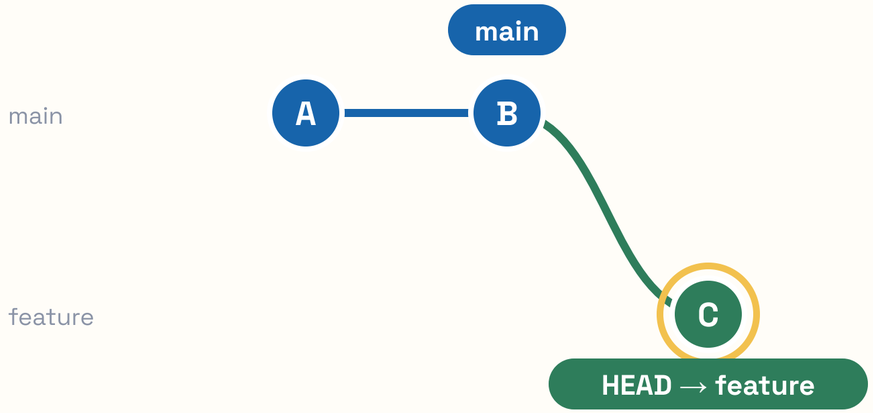
PICTURE IT BRANCHES

Creating a branch copies nothing

git switch -c feature



git commit (on feature)



A branch is just a label. Only the branch HEAD points to moves when you commit.

TRY IT BRANCHES

Create, commit, switch

```
$ git switch -c feature/coupons
Switched to a new branch 'feature/coupons'
$ echo "coupons" > coupons.txt
$ git add coupons.txt
$ git commit -m "Add coupons"
[feature/coupons 93b1d0e] Add coupons
 1 file changed, 1 insertion(+)
 create mode 100644 coupons.txt
$ git log --oneline --all --graph
* 93b1d0e Add coupons
* cbae48b Add home page
$ git switch main
Switched to branch 'main'
$ git ls-files
home.txt
```

1. **switch -c**: create + switch in one step.
2. main doesn't know about coupons.txt yet.
3. Switching branches changes the files in your folder.

TRY IT MERGING

Fast-forward: main didn’t move

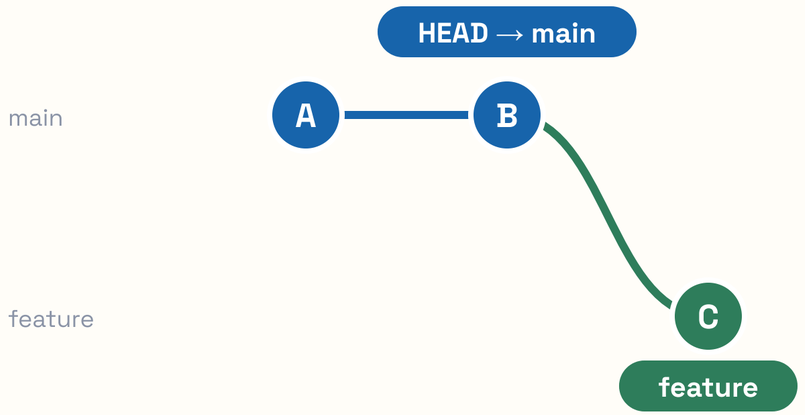
```
$ git switch main
Switched to branch 'main'
$ git merge feature/search
Updating cbae48b..bed5ce2
Fast-forward
 search.txt | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 search.txt
$ git log --oneline --graph --all
* bed5ce2 Add search
* cbae48b Add home page
```

1. Git just slides main forward.
2. No merge commit: history stays a straight line.

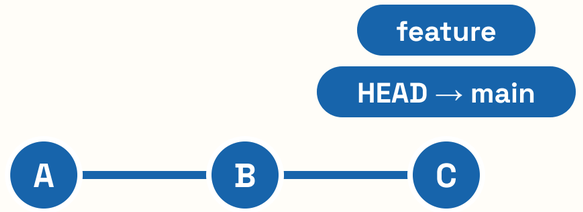
PICTURE IT FAST-FORWARD

Fast-forward merge

Before



`git merge feature`



main had no new commits, so Git simply **slides the label forward**. No merge commit.

TRY IT MERGING

Three-way: both branches moved

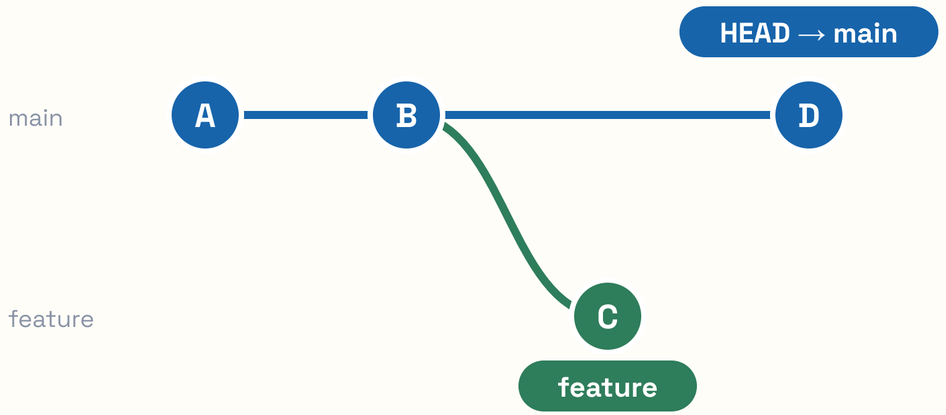
```
$ git switch main
Switched to branch 'main'
$ echo "footer" > footer.txt
$ git add footer.txt
$ git commit -qm "Add footer"
$ git merge --no-edit feature/search
Merge made by the 'ort' strategy.
 search.txt | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 search.txt
$ git log --oneline --graph --all
*   28c942c Merge branch 'feature/search'
|\
| * bed5ce2 Add search
* | 9e81205 Add footer
|/
* cbae48b Add home page
```

1. Git creates a **merge commit** with two parents.
2. The graph shows the two lines joining.
3. **--no-ff** forces a merge commit even when a fast-forward is possible.

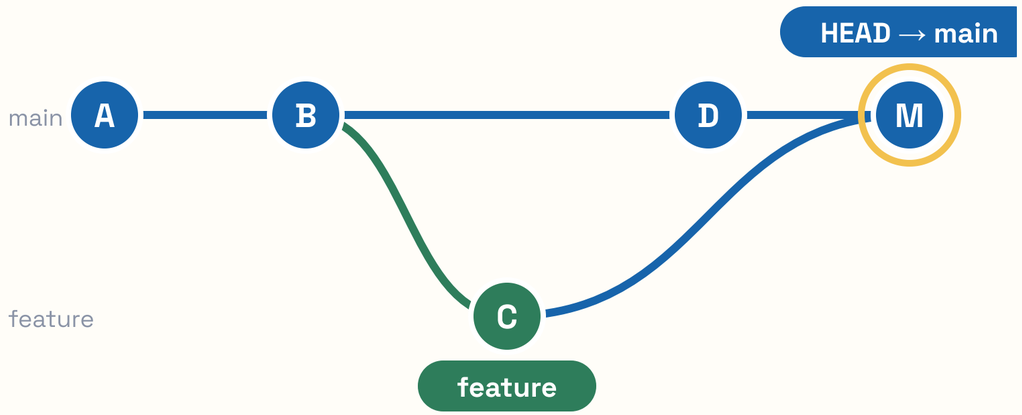
PICTURE IT THREE-WAY MERGE

Three-way merge

Before



`git merge feature`



Both sides have new work, so Git creates **M**, a merge commit with **two parents**.

TRY IT CONFLICTS

Both branches changed the same line

```
$ git merge feature/sale
Auto-merging banner.txt
CONFLICT (content): Merge conflict in banner.txt
Automatic merge failed; fix conflicts and then commit the result.
$ git status --short
UU banner.txt
$ cat banner.txt
<<<<<<< HEAD
Welcome! Free shipping today
=======
Big SALE: 20% off
>>>>>>> feature/sale
```

1. **UU** = unmerged: Git needs you.
2. Between **<<<<<<<** and **=======**: your branch (HEAD).
3. Between **=======** and **>>>>>>>**: theirs.

STEP BY STEP CONFLICTS

Resolving a conflict in 4 steps

STEP	DO
1 Find	`git status` : every UU file
2 Decide	edit the file to the final content (keep one side, the other, or a mix); delete the markers
3 Mark resolved	`git add banner.txt`
4 Finish	`git commit` (run the tests first!)
or cancel	`git merge --abort`

> VS Code shows buttons: Accept Current / Incoming / Both. Still read the result.

COMMON MISTAKE CONFLICTS

Committing conflict markers

✗ WRONG

```
git add .
git commit -m "Merge sale"
# banner.txt now contains:
# <<<<<<< HEAD ... ======= ... >>>>>>>
```

The site shows raw markers, or the code doesn't even compile. It happens when you **git add .** without looking.

✓ RIGHT

```
git diff --check        # finds leftover markers
git grep -n "<<<<<<<"
# fix, test, then add and commit
```

Check for markers and run the tests before committing a resolution.

IN REAL JOBS BRANCHES

Branching at work

One branch per ticket

```
git switch -c feature/SHOP-142-coupons
```

Keep up with main

```
git fetch
git merge origin/main      # or rebase (Module 5)
```

Merge through a pull request

```
push the branch -> open PR -> review -> merge
```

Tidy up

```
git branch --merged main
git branch -d feature/SHOP-142-coupons
```

INTERVIEW SAY THIS IN THE INTERVIEW

“How do you resolve a merge conflict?”

- Conflicts happen when both branches changed the **same lines**.
- **git status** lists the conflicted files; open each one and decide the final content.
- Remove the **<<<<<<< ======= >>>>>>>** markers, **git add**, then **git commit**.
- Run the tests; **git merge --abort** cancels everything.
- Avoid them: small branches, merge main often.

> **Follow-up they ask next:** ”Fast-forward vs three-way merge?” (no new commits on the target: just move the pointer; otherwise a merge commit with two parents).

RECAP REMEMBER THESE

Module 4 in 5 lines

1. A branch = a movable pointer; HEAD = where you are.
2. **git switch -c** to create, **git switch** to move.
3. Fast-forward: just move the pointer. Three-way: a merge commit.
4. Conflicts: edit, remove markers, **add**, **commit** (or **--abort**).
5. Delete merged branches with **git branch -d**.

PRACTICE NOW

Do exercises/ex04_merge_conflict/: merge dark mode and resolve the conflict.